

Hochschule Karlsruhe
Fakultät Maschinenbau und Mechatronik
Modul: Künstliche Intelligenz in der Produktion

State-of-the-Art-Recherche

3D Keypoint Detection

Verfahren, Datensätze, Repositories und Umsetzungshinweise

Projektkontext

Erkennen von Keypoints an Objekten, die durch 3D-Punktwolken repräsentiert werden. Die erkannten Keypoints können anschließend für Aufgaben wie Pose Detection, Visual Servoing, Shape Matching, Objektregistrierung oder Objektklassifikation genutzt werden.

Projektgruppe: _____

Bearbeitende: _____

Dozent/in: _____

Abgabedatum: 14. Mai 2026

Zusammenfassung

Diese Dokumentation dient als strukturierte Einarbeitung in ausgewählte Verfahren der 3D Keypoint Detection. Im Fokus stehen Methoden, für die neben der wissenschaftlichen Veröffentlichung auch ein öffentliches Code-Repository verfügbar ist. Dadurch soll die spätere praktische Umsetzung, das Nachtrainieren auf KeypointNet und die Evaluation im Rahmen des Projekts vorbereitet werden. Die Darstellung ist bewusst erklärend gehalten: Zu jedem Verfahren werden Problemstellung, Grundidee, Modellarchitektur, Trainingsprinzip, Datennutzung, Repository und mögliche Projektrelevanz beschrieben. Die Dokumentation ist keine vollständige systematische Literaturstudie, sondern eine fokussierte Wissensbasis zur begründeten Auswahl einer umsetzbaren Methode.

Inhaltsverzeichnis

1	Ausgangssituation und Zielsetzung	1
2	Grundlagen der 3D Keypoint Detection	1
2.1	Was sind 3D-Keypoints?	1
2.2	Wichtige Eigenschaften guter Keypoints	1
2.3	Supervised, weakly supervised, self-supervised und unsupervised	2
3	Datengrundlage: KeypointNet	2
3.1	Kurzbeschreibung	2
3.2	Repository und Datenstruktur	3
3.3	Bedeutung für Training und Evaluation	3
4	3DFeat-Net: Weakly Supervised Local 3D Features for Point Cloud Registration	3
4.1	Repository und praktische Nutzung	3
4.2	Problemstellung	4
4.3	Grundidee des Verfahrens	5
4.4	Modellarchitektur und Training	5
4.5	Einordnung für das Projekt	5
5	UKPGAN: A General Self-Supervised Keypoint Detector	5
5.1	Repository und praktische Nutzung	5
5.2	Problemstellung	6
5.3	Grundidee des Verfahrens	7
5.4	Rotationinvariante Merkmale	7
5.5	Training und Verluste	7
5.6	Einordnung für das Projekt	7
6	Skeleton Merger: An Unsupervised Aligned Keypoint Detector	8
6.1	Repository und praktische Nutzung	8
6.2	Problemstellung	8

6.3	Grundidee des Verfahrens	9
6.4	Composite Chamfer Distance	9
6.5	Ausrichtung und Semantik	9
6.6	Einordnung für das Projekt	9
7	SC3K: Self-supervised and Coherent 3D Keypoints Estimation	10
7.1	Repository und praktische Nutzung	10
7.2	Problemstellung	11
7.3	Grundidee des Verfahrens	11
7.4	Modellarchitektur und Verluste	11
7.5	Einordnung für das Projekt	12
8	KeypointDETR: An End-to-End 3D Keypoint Detector	12
8.1	Repository und praktische Nutzung	12
8.2	Problemstellung	12
8.3	Grundidee des Verfahrens	13
8.4	Transformer-Architektur	13
8.5	Training und Daten	13
8.6	Einordnung für das Projekt	13
9	FL3K: A Fast and Lightweight 3D Keypoint Detector	14
9.1	Repository und praktische Nutzung	14
9.2	Problemstellung	14
9.3	Grundidee des Verfahrens	14
9.4	Einordnung für das Projekt	15
10	Methodische Auswahl für die praktische Umsetzung	15
10.1	Variante A: Skeleton Merger als Hauptmethode	15
10.2	Variante B: SC3K als moderne selbstüberwachte Methode	15
10.3	Variante C: KeypointDETR als moderner supervised Ansatz	15
10.4	Empfohlene pragmatische Vorgehensweise	16
11	Evaluation und Bewertungskriterien	16
11.1	Qualitative Evaluation	16
11.2	Quantitative Evaluation	16
11.3	Reproduzierbarkeit	17
12	Fazit	17

1 Ausgangssituation und Zielsetzung

Die Projektaufgabe behandelt die Erkennung von 3D-Keypoints an Objekten, die als Punktwolken vorliegen. Als Datensatz ist KeypointNet vorgegeben. Zusätzlich werden in der Aufgabenstellung UKPGAN und Skeleton Merger als mögliche Modelle genannt. Die Projektbewertung legt nicht den Schwerpunkt auf eine vollständige Neuentwicklung einer eigenen Architektur, sondern auf das Verständnis, die Umsetzung, Anpassung und Validierung verfügbarer wissenschaftlicher Ansätze.

Daraus ergeben sich für diese Dokumentation drei Ziele. Erstens soll ein gemeinsames Grundverständnis von 3D Keypoint Detection geschaffen werden. Zweitens sollen die ausgewählten Paper so zusammengefasst werden, dass die jeweilige Methode nachvollziehbar wird. Drittens soll zu jedem Verfahren geklärt werden, ob und wie das zugehörige öffentliche Repository für ein Nachtraining oder eine Evaluation genutzt werden kann.

Die betrachteten Methoden sind:

- KeypointNet als Datengrundlage,
- 3DFeat-Net als ältere, schwach überwachte Grundlage für gelernte 3D-Features,
- UKPGAN als selbstüberwachter Keypoint-Detektor,
- Skeleton Merger als unüberwachter, skeletonbasierter Detektor,
- SC3K als moderner selbstüberwachter Ansatz für robuste und kohärente Keypoints,
- KeypointDETR als aktueller Transformer-basierter End-to-End-Detektor,
- FL3K als schneller und leichter saliencybasierter 3D-Keypoint-Detektor.

Die Übersicht ist bewusst begrenzt. Für das eigentliche Projekt soll am Ende nicht eine große Anzahl von Verfahren implementiert werden. Realistisch ist die vertiefte Umsetzung eines Hauptverfahrens und gegebenenfalls einer einfachen Baseline oder eines Vergleichsverfahrens.

2 Grundlagen der 3D Keypoint Detection

2.1 Was sind 3D-Keypoints?

3D-Keypoints sind ausgewählte Punkte auf oder nahe einer 3D-Objektoberfläche, die eine besondere geometrische, semantische oder funktionale Bedeutung besitzen. Beispiele sind die Flügelspitzen eines Flugzeugs, die Beine eines Stuhls, der Griff einer Tasse oder markante Ecken und Übergänge an einem Objekt. Im Gegensatz zu einer vollständigen Punktwolke stellen Keypoints eine stark verdichtete Objektbeschreibung dar.

Diese Verdichtung ist nützlich, weil viele nachgelagerte Aufgaben nicht jeden einzelnen Punkt benötigen. Für Registrierung, Pose-Schätzung, Shape Matching oder Objektverfolgung reicht häufig eine kleine Menge stabiler, wiedererkennbarer Punkte. Ein guter Keypoint-Detektor soll daher nicht nur auffällige Punkte finden, sondern möglichst stabile, wiederholbare und semantisch sinnvolle Punkte liefern.

2.2 Wichtige Eigenschaften guter Keypoints

Für das Projekt sind mehrere Eigenschaften relevant:

Lokalisierungsgenauigkeit Der vorhergesagte Keypoint sollte nahe an der gesuchten Position liegen.

Wiederholbarkeit Dasselbe Objekt sollte unter Transformationen, Rauschen oder unterschiedlicher Punktdichte ähnliche Keypoints liefern.

Semantische Kohärenz Bei Objekten derselben Kategorie sollte ein Keypoint-Index möglichst dieselbe Objektregion beschreiben, zum Beispiel immer die linke Flügelspitze.

Oberflächennähe Keypoints sollten auf oder nahe der Punktwolkenoberfläche liegen und nicht im freien Raum.

Abdeckung Die Keypoints sollten nicht alle auf einer kleinen Region liegen, sondern relevante Teile des Objekts erfassen.

Praktische Umsetzbarkeit Für das Projekt ist wichtig, dass Code, Datenformat, Training und Evaluation nachvollziehbar verfügbar sind.

2.3 Supervised, weakly supervised, self-supervised und unsupervised

Die betrachteten Verfahren unterscheiden sich stark darin, welche Art von Trainingsinformation verwendet wird. Supervised Ansätze nutzen menschlich annotierte Keypoints als Zielwerte. Das ist besonders naheliegend bei KeypointNet, weil dort semantische 3D-Keypoints vorhanden sind. Self-supervised und unsupervised Ansätze verzichten dagegen auf direkte Keypoint-Labels und formulieren Hilfsaufgaben, beispielsweise Rekonstruktion, Konsistenz unter Rotation oder Stabilität unter Störungen. Weakly supervised Methoden nutzen schwächere Informationen, etwa ob zwei Punktwolken denselben Ort oder einen starken Überlappungsbereich darstellen.

Diese Unterscheidung ist für die spätere Methodenauswahl wichtig. Ein supervised Ansatz kann direkt aus KeypointNet lernen, benötigt aber saubere Annotationen und eine passende Kategorieauswahl. Ein self-supervised oder unsupervised Ansatz ist methodisch flexibler, muss aber durch geeignete Metriken und Visualisierungen nachweisen, dass die gefundenen Keypoints tatsächlich sinnvoll sind.

3 Datengrundlage: KeypointNet

3.1 Kurzbeschreibung

GitHub-Repository: <https://github.com/qq456cvb/KeypointNet>

KeypointNet ist die zentrale Datengrundlage des Projekts. Der Datensatz enthält 83.231 Keypoints und 8.329 3D-Modelle aus 16 Objektkategorien. Die Modelle basieren auf ShapeNet und wurden über viele menschliche Annotationen mit semantischen Keypoints versehen [1]. Im Repository werden außerdem bereits gesampelte Punktwolken mit 2048 Punkten pro ShapeNet-Modell bereitgestellt.

Die 16 Kategorien umfassen unter anderem airplane, bathtub, bed, bottle, cap, car, chair, guitar, helmet, knife, laptop, motorcycle, mug, skateboard, table und vessel. Für das Projekt ist eine Einschränkung auf wenige Kategorien sinnvoll, etwa airplane, chair, car oder mug. Dadurch bleibt das Training überschaubar und die Ergebnisse lassen sich besser interpretieren.

3.2 Repository und Datenstruktur

Das offizielle Repository beschreibt die Datenstruktur im Wesentlichen über Annotationen und Punktwolken. Die annotierten JSON-Dateien liegen unter **annotations**; die gesampelten Punktwolken liegen unter **pcds**. Zusätzlich werden Links zu Google Drive und OneDrive angeboten.

```
KeypointNet/
|-- annotations/
|   |-- airplane.json
|   |-- chair.json
|   '-- ...
|-- pcds/
|   |-- 02691156/
|       |-- <model_id>.pcd
|       '-- ...
|   '-- ...
'-- README.md
```

Die JSON-Annotationen enthalten unter anderem eine Klassen-ID, eine Modell-ID und Keypoint-Koordinaten. Für viele Trainingspipelines müssen diese Daten noch in das Format des jeweiligen Modell-Repositories gebracht werden. SC3K erwartet zum Beispiel zusätzlich Splits und vorab generierte Posen. KeypointDETR benötigt geodesische Distanzkarten, aus denen Heatmaps für das Training erzeugt werden.

3.3 Bedeutung für Training und Evaluation

KeypointNet eignet sich besonders gut, wenn ein Verfahren semantische Keypoints lernen oder evaluieren soll. Für supervised Verfahren wie KeypointDETR können die Keypoint-Annotationen direkt in Trainingsziele umgewandelt werden. Für self-supervised oder unsupervised Verfahren wie UKPGAN, Skeleton Merger oder SC3K können die Annotationen dagegen primär zur Evaluation genutzt werden. Das Modell lernt dann nicht direkt aus den Labels, aber seine vorhergesagten Punkte können später mit den menschlichen Keypoints verglichen werden.

Für eine wissenschaftlich nachvollziehbare Projektdurchführung sollte früh festgelegt werden:

- welche Objektkategorien verwendet werden,
- wie Trainings-, Validierungs- und Testdaten getrennt werden,
- wie viele Keypoints pro Kategorie betrachtet werden,
- ob canonical poses oder zufällige Rotationen genutzt werden,
- welche Metriken zur Bewertung verwendet werden.

4 3DFeat-Net: Weakly Supervised Local 3D Features for Point Cloud Registration

4.1 Repository und praktische Nutzung

GitHub-Repository: <https://github.com/yewzijian/3DFeatNet>

Das Repository enthält eine TensorFlow-1-Implementierung von 3DFeat-Net. Die Struktur ist stärker auf Punktwolken-Registrierung und LiDAR-Daten ausgerichtet als auf KeypointNet. Wichtige Bestandteile sind:

```
3DFeatNet/  
|-- data/  
|-- docs/  
|-- example_data/  
|-- models/  
|   '-- feat3dnet.py  
|-- scripts/  
|-- scripts_data_processing/  
|-- tf_ops/  
|-- train.py  
|-- train.sh  
|-- inference.py  
'-- inference_example.sh
```

Vor der Nutzung müssen die angepassten TensorFlow-Operationen in `tf_ops` kompiliert werden. Die Autoren geben als ursprüngliche Umgebung Python 3.5, TensorFlow 1.4, CUDA 8.0, NumPy 1.13.3 und scikit-learn 0.19.1 an. Für Training und Evaluation werden außerdem MATLAB-Skripte verwendet. Das Training ist im Repository über `train.sh` organisiert und erfolgt in zwei Stufen: zunächst wird das Descriptor-Subnetz trainiert, anschließend das vollständige Modell mit Rotation und Attention. Das Training ist laut Repository für die bereitgestellten Daten eher auf Oxford-RobotCar-artige Daten ausgelegt und dauert etwa 1 bis 1,5 Tage, bis es sättigt.

Für unser Projekt ist 3DFeat-Net deshalb eher als konzeptionelle Grundlage und weniger als primärer Umsetzungskandidat geeignet. Eine direkte Nutzung mit KeypointNet wäre möglich, würde aber eine stärkere Anpassung der Datenpipeline erfordern als bei UKPGAN, Skeleton Merger, SC3K oder KeypointDETR.

4.2 Problemstellung

3DFeat-Net adressiert das Problem der Punktwolken-Registrierung. Bei der Registrierung sollen zwei Punktwolken, die denselben oder überlappende Teile eines 3D-Szenarios darstellen, räumlich zueinander ausgerichtet werden. Dafür werden zunächst Korrespondenzen zwischen Punkten benötigt. Klassische Verfahren verwenden dafür handgefertigte Keypoint-Detektoren und Deskriptoren. 3DFeat-Net versucht dagegen, Detektor und Descriptor gemeinsam zu lernen [2].

Der Ansatz ist schwach überwacht. Das bedeutet, dass keine manuell annotierten korrespondierenden Punkte benötigt werden. Stattdessen wird genutzt, ob zwei Punktwolken eine hohe oder niedrige Überlappung besitzen. Diese Information kann aus GPS/INS-getaggten Daten abgeleitet werden.

4.3 Grundidee des Verfahrens

Die Kernidee lautet: Ein guter 3D-Keypoint ist ein Punkt, an dem ein lokaler Descriptor entstehen kann, der zuverlässig zwischen überlappenden Punktwolken matchbar ist. Keypoint Detection wird in 3DFeat-Net daher nicht isoliert betrachtet, sondern im Zusammenhang mit Feature Matching und Registrierung.

Das Modell nutzt eine Siamese-Architektur. Zwei Punktwolken werden durch dasselbe Netzwerk verarbeitet. Aus lokalen Regionen werden Descriptoren berechnet. Über einen Triplet Loss lernt das Netzwerk, dass überlappende Punktwolken ähnliche lokale Features und nicht überlappende Punktwolken unterscheidbare Features erhalten. Eine Attention-Schicht gewichtet dabei, welche lokalen Punkte besonders wichtig sind. Diese Attention-Werte werden zur Ableitung von Keypoint-Salienzen genutzt.

4.4 Modellarchitektur und Training

3DFeat-Net baut auf PointNet-artiger Verarbeitung von Punktwolken auf. Die Eingangspunktwolke wird in lokale Cluster bzw. Regionen zerlegt. Für diese Regionen werden Feature-Vektoren erzeugt. Die Attention-Komponente bewertet, welche Regionen für die Unterscheidung und Zuordnung besonders hilfreich sind. Während des Trainings werden Triplets aus Anchor-, Positive- und Negative-Beispielen verwendet. Positive Beispiele stammen aus räumlich nahen bzw. stark überlappenden Bereichen, negative Beispiele aus weit entfernten Bereichen.

Wichtig ist, dass die Keypoints hier nicht direkt als semantische Objektpunkte gelernt werden. Die Keypoints entstehen als Nebenprodukt eines Registrierungsziels. Der Ansatz eignet sich daher gut zur Erklärung, wie gelernte 3D-Keypoint-Detektion historisch aus Descriptor-Learning und Registrierung hervorgegangen ist.

4.5 Einordnung für das Projekt

3DFeat-Net ist für die Dokumentation wertvoll, weil es zeigt, wie schwache Supervision für 3D-Feature-Learning genutzt werden kann. Für die konkrete Umsetzung auf KeypointNet ist es jedoch weniger naheliegend. Der Grund ist, dass KeypointNet objektzentrierte Shape-Modelle mit semantischen Keypoints enthält, während 3DFeat-Net auf Outdoor-LiDAR-Registrierung und Überlappungsinformation ausgelegt ist. Wenn nur eine Methode sauber nachtrainiert werden soll, ist 3DFeat-Net daher eher als Hintergrundpapier einzuordnen.

5 UKPGAN: A General Self-Supervised Keypoint Detector

5.1 Repository und praktische Nutzung

GitHub-Repository: <https://github.com/q456cvb/UKPGAN>

Das Repository enthält die offizielle Implementierung von UKPGAN. Der Hauptzweig ist eine TensorFlow-Implementierung; zusätzlich verweist das Repository auf einen PyTorch-Branch. Die sichtbare Repository-Struktur enthält unter anderem:

UKPGAN/

```
|-- config/  
|-- data/  
|   '-- 3DMatch/kitchen/  
|-- images/  
|-- sdv_src/  
|-- dataflow.py  
|-- environment.yml  
|-- eval_iou.py  
|-- model.py  
|-- ops.py  
|-- train.py  
|-- utils.py  
'-- visualize.py
```

Für die TensorFlow-Version wird ein Conda-Environment über `environment.yml` erstellt. Zusätzlich müssen die Smoothed-Density-Value-Komponenten in `sdv_src` kompiliert werden. Dafür werden Pybind11 und PCL benötigt. Für das Training auf ShapeNet- bzw. KeypointNet-Punktwolken sollen die Punktwolken aus KeypointNet heruntergeladen und der `pcds`-Ordner in das Repository gelegt werden. Anschließend wird in `config/config.yaml` die Kategorie, beispielsweise `chair`, gesetzt. Ein typischer Trainingsaufruf ist:

```
visdom -port 1080  
python train.py cat_name=chair
```

Zur Evaluation mit menschlichen Keypoint-Annotationen wird im Repository `eval_iou.py` bereitgestellt. Für Visualisierung kann `visualize.py` verwendet werden, beispielsweise mit Non-Maximum-Suppression:

```
python eval_iou.py --kpnet_root /path/to/keypointnet  
python visualize.py --type shapenet --nms --nms_radius 0.1
```

5.2 Problemstellung

UKPGAN behandelt 3D Keypoint Detection als selbstüberwachtes Problem. Das bedeutet, dass während des Trainings keine menschlichen Keypoint-Annotationen benötigt werden. Stattdessen soll das Modell selbst lernen, welche Punkte für die Beschreibung eines Objekts besonders informativ sind. Die Autoren formulieren Keypoint Detection als eine Form von Informationskompression: Eine kleine Menge von Keypoints soll ausreichend Information enthalten, um die ursprüngliche Objektform zu rekonstruieren [3].

Dieser Gedanke ist für das Projekt gut verständlich. Wenn ein Modell aus wenigen Keypoints die Form eines Objekts wiederherstellen kann, müssen diese Keypoints offenbar wichtige Strukturinformationen tragen. Im Gegensatz zu rein lokalen geometrischen Detektoren sollen die gefundenen Punkte nicht nur auffällig, sondern auch für die Formbeschreibung relevant sein.

5.3 Grundidee des Verfahrens

UKPGAN besteht aus einem Encoder-Decoder-Prinzip. Der Encoder verarbeitet eine Punktwolke und sagt für jeden Punkt eine Keypoint-Wahrscheinlichkeit sowie zusätzliche semantische Einbettungen vorher. Aus diesen Wahrscheinlichkeiten wird eine kleine Menge salienter Punkte abgeleitet. Der Decoder versucht anschließend, aus diesen komprimierten Informationen die ursprüngliche Punktwolke zu rekonstruieren.

Ein zentrales Element ist die Kontrolle der Sparsity. Wenn zu viele Punkte als Keypoints ausgewählt werden, ist die Aufgabe trivial, weil nahezu die gesamte Punktwolke erhalten bleibt. Wenn zu wenige oder falsche Punkte gewählt werden, gelingt die Rekonstruktion nicht. UKPGAN nutzt deshalb eine GAN-basierte Sparsity-Kontrolle, um die Zahl und Verteilung der Keypoints zu regulieren.

5.4 Rotationinvariante Merkmale

Ein praktisches Problem bei 3D-Punktwolken ist die Abhängigkeit vom Koordinatensystem. Dasselbe Objekt kann rotiert oder verschoben vorliegen. UKPGAN adressiert dies über lokale rotationsinvariante Merkmale. Dazu wird für lokale Nachbarschaften ein Local Reference Frame geschätzt. Die Punkte in einer lokalen Umgebung werden in dieses lokale Koordinatensystem transformiert und anschließend in einer Smoothed-Density-Value-Repräsentation beschrieben. Dadurch soll die Detektion unter starren Transformationen stabiler werden.

Diese Eigenschaft unterscheidet UKPGAN von Ansätzen, die stark auf eine feste kanonische Objektpose angewiesen sind. Für das Projekt kann dies relevant sein, wenn Punktwolken nicht immer exakt gleich ausgerichtet sind.

5.5 Training und Verluste

Das Training kombiniert mehrere Ideen. Die Rekonstruktionskomponente sorgt dafür, dass die Keypoints hinreichend informativ sind. Die Sparsity-Komponente verhindert, dass zu viele Punkte ausgewählt werden. Die semantischen Einbettungen sollen zudem helfen, konsistente und aussagekräftige Keypoint-Repräsentationen zu erzeugen.

Da keine direkten Keypoint-Labels benötigt werden, kann UKPGAN auf sauberen Objektkollektionen trainiert werden. Die Autoren zeigen außerdem Anwendungen auf nicht-rigiden Körpern und realen Szenen. Für das Projekt ist vor allem die Nutzung mit KeypointNet bzw. ShapeNet-Modellen relevant, weil diese Daten bereits in der Aufgabenstellung vorgesehen sind.

5.6 Einordnung für das Projekt

UKPGAN ist ein guter Kandidat, wenn ein selbstüberwachtes Verfahren umgesetzt werden soll. Es passt zur Aufgabenstellung, weil es direkt in der Aufgabenfolie genannt wird und ein offizielles Repository mit Trainings- und Evaluationsskripten existiert. Der methodische Kern ist nachvollziehbar: Keypoints werden als komprimierte Formrepräsentation gelernt. Gleichzeitig ist die Implementierung nicht trivial, weil die TensorFlow-Version ältere Abhängigkeiten und eine

kompilierte SDV-Komponente benötigt. Der PyTorch-Branch kann die praktische Umsetzung erleichtern, muss aber gesondert geprüft werden.

6 Skeleton Merger: An Unsupervised Aligned Keypoint Detector

6.1 Repository und praktische Nutzung

GitHub-Repository: <https://github.com/eliphatfs/SkeletonMerger>

Das Repository ist vergleichsweise übersichtlich aufgebaut. Es enthält eine PyTorch-Implementierung und mehrere Skripte für Training, Prediction und Evaluation. Die README beschreibt folgende wichtige Bestandteile:

```
SkeletonMerger/  
|-- _readme_resources/  
|-- merger/  
|   |-- pointnetpp/  
|   |-- composed_chamfer.py  
|   |-- data_flow.py  
|   '-- merger_net.py  
|-- eval_keypointnet.py  
|-- predictor_keypointnet.py  
|-- train.py  
'-- README.md
```

`merger/pointnetpp` enthält eine angepasste PointNet++-Implementierung. `composed_chamfer.py` enthält die effiziente Implementierung der Composite Chamfer Distance. `data_flow.py` ist für Datenladen und Vorverarbeitung zuständig. `merger_net.py` enthält die eigentliche Skeleton-Merger-Architektur.

Das Repository stellt Hilfeinformationen über die Option `-h` der Skripte bereit. Ein vortrainiertes Modell für Stühle aus ShapeNetCore wird angeboten. Die Autoren empfehlen dennoch, bei eigener Datennutzung lokal neu zu trainieren, weil Achsenreihenfolge, Skalierung und Orientierung zwischen Datensätzen variieren können. Laut Repository liefert Skeleton Merger oft bereits nach 5 bis 10 Epochen brauchbare Ergebnisse; für die volle Leistung werden eher 50 bis 100 Epochen und eine Auswahl anhand Validierungsfehler oder Downstream-Aufgabe empfohlen.

6.2 Problemstellung

Skeleton Merger adressiert ein Problem, das bei vielen unüberwachten Keypoint-Detektoren auftritt. Sie finden zwar geometrisch auffällige Punkte, aber diese Punkte sind häufig nicht geordnet, nicht semantisch ausgerichtet und decken das Objekt nicht gut ab. Für viele Anwendungen reicht es jedoch nicht, irgendeine Menge auffälliger Punkte zu finden. Wenn verschiedene Objekte derselben Kategorie verglichen werden sollen, ist wichtig, dass Keypoint 1, Keypoint 2 usw. möglichst konsistente Objektregionen beschreiben.

Skeleton Merger versucht daher, nicht nur Keypoints, sondern ein geordnetes Skelett des Objekts zu lernen. Die Keypoints bilden dabei Knoten eines Graphen; die Kanten zwischen ihnen beschreiben eine grobe Objektstruktur [4].

6.3 Grundidee des Verfahrens

Die Intuition ist anschaulich: Eine kleine Menge isolierter Keypoints kann schwer interpretierbar sein. Werden diese Punkte jedoch durch Kanten zu einem Skelett verbunden, entsteht eine strukturierte Repräsentation, die die Objektform besser beschreibt. Bei einem Stuhl könnten Keypoints beispielsweise Sitzfläche, Lehne und Beine markieren, während die Kanten die grobe Stuhlstruktur verbinden.

Das Verfahren nutzt eine Autoencoder-Architektur. Der Encoder schlägt Keypoints vor und sagt Aktivierungsstärken für mögliche Kanten zwischen diesen Keypoints vorher. Daraus entsteht ein Skelett, also ein Graph aus Punkten und Kanten. Der Decoder sampelt Punkte entlang der aktivierten Skelettkanten, verfeinert diese durch Offsets zu kleinen Sub-Punktwolken und setzt diese Sub-Punktwolken wieder zusammen. Ziel ist, die ursprüngliche Eingabepunktwolke möglichst gut aus dem Skelett zu rekonstruieren.

6.4 Composite Chamfer Distance

Für die Rekonstruktion wird eine angepasste Distanzfunktion benötigt. Eine normale Chamfer Distance misst, wie gut zwei Punktmengen räumlich zueinander passen. Skeleton Merger verwendet eine Composite Chamfer Distance. Diese unterscheidet zwischen Fidelity und Coverage. Fidelity bedeutet, dass rekonstruierte Punkte nahe an der Eingabe liegen sollen. Coverage bedeutet, dass die Eingabepunktwolke durch die rekonstruierten Sub-Punktwolken möglichst gut abgedeckt werden soll.

Diese Verlustfunktion passt zur Idee des Verfahrens. Es reicht nicht, einzelne Punkte gut zu treffen; das Skelett soll die gesamte Objektstruktur sinnvoll abdecken. Gleichzeitig verhindern die Aktivierungsstärken der Kanten, dass alle möglichen Kanten gleichermaßen genutzt werden.

6.5 Ausrichtung und Semantik

Ein wichtiger Punkt ist die Ordnung der Keypoints. Da der Encoder eine feste Anzahl von Keypoint-Proposals erzeugt und die Kantenstruktur über feste mögliche Verbindungen modelliert, entsteht stärker geordnete Struktur als bei Verfahren, die nur eine ungeordnete Punktmenge ausgeben. Genau dadurch ist Skeleton Merger für KeypointNet interessant: KeypointNet enthält semantische Keypoints, und Skeleton Merger versucht, ohne direkte Supervision semantisch sinnvolle und ausgerichtete Punkte zu finden.

Im Paper wird berichtet, dass Skeleton Merger auf KeypointNet mit überwachten Verfahren vergleichbare Ergebnisse erreichen kann und robust gegenüber Rauschen und Subsampling ist [4]. Für die Projektpräsentation ist diese Methode besonders gut erklärbar, weil sich die Rekonstruktionsidee über ein Skelett visuell anschaulich darstellen lässt.

6.6 Einordnung für das Projekt

Skeleton Merger ist wahrscheinlich einer der praktisch stärksten Kandidaten für die Umsetzung. Es ist direkt in der Aufgabenstellung genannt, besitzt ein öffentliches Repository, arbeitet mit KeypointNet und die Kernidee ist gut nachvollziehbar. Im Gegensatz zu UKPGAN steht hier

nicht die allgemeine Informationskompression über sparse Keypoints im Vordergrund, sondern die Rekonstruktion über eine graphartige Skelettstruktur. Dadurch wird der Unterschied zu UKPGAN deutlich, ohne dass ein direkter Leistungsvergleich notwendig ist.

7 SC3K: Self-supervised and Coherent 3D Keypoints Estimation

7.1 Repository und praktische Nutzung

GitHub-Repository: <https://github.com/IIT-PAVIS/SC3K>

Das SC3K-Repository ist auf KeypointNet und eine reproduzierbare Trainingsstruktur ausgelegt. Die sichtbare Struktur enthält unter anderem:

```
SC3K/
|-- config/
|-- dataset/
|-- images/
|-- data_loader.py
|-- metrics.py
|-- network.py
|-- sc3k_environment.yml
|-- train.py
|-- test.py
|-- utils.py
'-- visualizations.py
```

Die Autoren empfehlen die Nutzung eines Conda-Environments über `sc3k_environment.yml`. Getestet wurde der Code unter anderem mit Python 3.6.12, Torch 1.10.1 und Torchvision 0.11.2. Training und Inferenz werden über `config/config.yaml` gesteuert. Für das Training werden beispielsweise `split: train`, `task: generic`, `batch_size` und `class_name` gesetzt. Anschließend wird trainiert mit:

```
python train.py
```

Die besten Gewichte werden laut Repository unter `train/<class_name>/Best_<class_name>_10kp.pth` gespeichert. Für die Inferenz wird `split: test` gesetzt und `best_model_path` auf die gespeicherten Gewichte gesetzt. Danach erfolgt der Aufruf:

```
python test.py
```

Die erwartete Datenstruktur ist ebenfalls dokumentiert:

```
dataset/
|-- annotation/
|   '-- airplane.json
|-- pcds/
|   '-- 02691156/
|       '-- *.pcd
|-- poses/
|   '-- 02691156/
```

```
|      '-- *.npz  
'-- splits/  
    |-- train.txt  
    |-- val.txt  
    '-- test.txt
```

Damit ist SC3K aus Umsetzungssicht interessant, weil Training, Test und Datenstruktur relativ klar beschrieben sind.

7.2 Problemstellung

SC3K betrachtet 3D-Keypoint-Detection in realistischeren Bedingungen. Punktwolken können rotiert, verrauscht und ausgedünnt sein. Das Verfahren soll Keypoints ohne direkte Annotationen finden, die dennoch robust, semantisch kohärent, kompakt und gut verteilt sind [5].

Der Begriff „coherent“ ist hier zentral. Es geht nicht nur darum, stabile Punkte zu finden. Die Reihenfolge bzw. Identität der Keypoints soll zwischen Objekten derselben Kategorie möglichst erhalten bleiben. Wenn Keypoint 3 bei einem Flugzeug die linke Flügelspitze beschreibt, sollte er diese Bedeutung auch bei anderen Flugzeugen behalten.

7.3 Grundidee des Verfahrens

SC3K erzeugt während des Trainings zwei zufällig rotierte Versionen derselben Punktwolke. Beide Versionen werden durch dasselbe Modell verarbeitet und liefern jeweils eine Menge von Keypoints. Da die verwendete Rotation bekannt ist, kann geprüft werden, ob die vorhergesagten Keypoints nach Rücktransformation konsistent zueinander sind.

Zusätzlich werden verschiedene Hilfsverluste eingesetzt, die gewünschte Eigenschaften erzwingen. Die Keypoints sollen nahe an der Oberfläche liegen, sich nicht gegenseitig überlappen, das Objekt gut abdecken und unter Transformationen konsistent bleiben. Außerdem wird als Proxy-Aufgabe die relative Pose zwischen den beiden Keypoint-Sets betrachtet. Dadurch lernt das Modell Keypoints, die für die räumliche Struktur des Objekts aussagekräftig sind.

7.4 Modellarchitektur und Verluste

Das Netzwerk verwendet einen PointNet-basierten Encoder zur Extraktion globaler Merkmale. Danach werden über Residualblöcke und eine Softmax-Struktur Wahrscheinlichkeiten für Keypoints geschätzt. Aus diesen Wahrscheinlichkeiten werden die eigentlichen 3D-Keypoint-Positionen berechnet.

Die Verlustfunktionen lassen sich in individuelle und gegenseitige Verluste einteilen. Individuelle Verluste bewerten ein einzelnes vorhergesagtes Keypoint-Set: Es soll nicht degenerieren, das Objekt abdecken und nahe an der Punktwolkenoberfläche liegen. Gegenseitige Verluste vergleichen die Keypoints zweier transformierter Versionen desselben Objekts. Dadurch werden Konsistenz und semantische Ordnung gefördert.

7.5 Einordnung für das Projekt

SC3K ist eine moderne Ergänzung zu UKPGAN und Skeleton Merger. Es ist aktueller und legt besonders viel Wert auf Robustheit gegenüber Rotation, Rauschen und reduzierter Punktdichte. Für eine Umsetzung ist das Repository gut geeignet, weil es ein klar beschriebenes Training mit KeypointNet-Struktur bereitstellt. Gegenüber Skeleton Merger ist die Methode weniger anschaulich über ein Skelett erklärbar, aber sie passt sehr gut, wenn Robustheit und Konsistenz unter Störungen im Projekt betont werden sollen.

8 KeypointDETR: An End-to-End 3D Keypoint Detector

8.1 Repository und praktische Nutzung

GitHub-Repository: <https://github.com/bibi547/KeypointDETR>

Das Repository enthält eine moderne PyTorch/PyTorch-Lightning-nahe Struktur. Sichtbare Bestandteile sind:

```
KeypointDETR/  
|-- config/  
|-- data/  
|-- models/  
|-- scripts/  
|   '-- geodesic_distance.py  
|-- utils/  
|-- pl_model.py  
|-- train.py  
|-- test.py  
'-- README.md
```

Vor dem Training müssen geodesische Distanzkarten erzeugt werden. Dafür stellt das Repository `scripts/geodesic_distance.py` bereit. Diese Distanzkarten werden genutzt, um Ground-Truth-Heatmaps aus den Keypoint-Annotationen zu erzeugen. Anschließend wird die Konfiguration in `config/keypoint_saliency.yaml` an Pfade, Dateinamen und Kategorien angepasst. Das Training und Testen erfolgt mit:

```
python train.py  
python test.py
```

Für ein Projekt ist hierbei wichtig, dass KeypointDETR stärker von den KeypointNet-Annotationen abhängt als die selbstüberwachten Verfahren. Die Datenvorbereitung ist dadurch aufwendiger, aber die Aufgabenstellung passt sehr direkt zum Datensatz.

8.2 Problemstellung

Viele ältere 3D-Keypoint-Verfahren erzeugen eine gemeinsame Heatmap über alle möglichen Keypoints. Aus dieser Heatmap müssen anschließend über Clustering oder Non-Maximum-Suppression die finalen Keypoints herausgefiltert werden. Diese Nachverarbeitung kann langsam

sein und hängt von Parametern ab. KeypointDETR formuliert 3D-Keypoint-Detection deshalb als End-to-End-Set-Prediction-Problem [6].

Das Modell soll nicht nur eine gemeinsame Saliency Map erzeugen, sondern mehrere Heatmaps und Wahrscheinlichkeiten für potenzielle Keypoints ausgeben. Jede Heatmap steht für einen möglichen Keypoint. Die zugehörige Wahrscheinlichkeit beschreibt, ob dieser Keypoint existiert und semantisch konsistent ist.

8.3 Grundidee des Verfahrens

KeypointDETR überträgt die DETR-Idee aus der Objektdetektion auf 3D-Keypoint-Detection. Bei DETR werden Objekte als Menge von Vorhersagen modelliert, die über bipartites Matching mit den Ground-Truth-Objekten verknüpft werden. KeypointDETR nutzt ein ähnliches Prinzip: Vorhergesagte Keypoint-Heatmaps und Wahrscheinlichkeiten werden mit Ground-Truth-Heatmaps gematcht. Das Matching erfolgt über den Hungarian Algorithmus.

Dadurch muss das Modell nicht vorab wissen, welche Prediction welchem Keypoint entspricht. Während des Trainings wird die beste Zuordnung zwischen Vorhersagen und Ground Truth gefunden. Das erlaubt eine flexiblere End-to-End-Optimierung als klassische Heatmap-Ansätze mit nachträglichem Clustering.

8.4 Transformer-Architektur

Die Architektur besteht aus einem point-wise Encoder und einem query-wise Decoder. Der Encoder verarbeitet die Punktwolke und erzeugt geometrische Features. Dafür werden dynamische Graphen im lokalen Feature-Raum genutzt und Self-Attention eingesetzt. So kann das Modell lokale Nachbarschaftsinformationen und größere Zusammenhänge in der Punktwolke erfassen.

Der Decoder arbeitet mit Keypoint-Queries. Diese Queries tauschen über Self-Attention Informationen aus und greifen über Cross-Attention auf die Punktfeatures zu. Dadurch kann das Modell Beziehungen zwischen Keypoints, ihren Positionen und ihren semantischen Eigenschaften modellieren. Das Ergebnis sind mehrere Heatmaps und Wahrscheinlichkeiten.

8.5 Training und Daten

KeypointDETR ist im Vergleich zu UKPGAN, Skeleton Merger und SC3K stärker supervised. Es benötigt Ground-Truth-Keypoints, aus denen Heatmaps erzeugt werden. KeypointNet ist dafür sehr passend. Die geodesischen Distanzkarten sind dabei wichtig, weil Distanzen auf der Objektoberfläche oft aussagekräftiger sind als reine euklidische Distanzen im Raum.

Für das Projekt bedeutet das: KeypointDETR kann theoretisch sehr sauber auf KeypointNet trainiert und evaluiert werden. Gleichzeitig ist die Vorbereitung anspruchsvoller, weil Distanzkarten, Heatmaps und Konfigurationen korrekt erzeugt werden müssen. Außerdem ist die Transformer-Architektur komplexer zu erklären als ein skeletonbasierter Autoencoder.

8.6 Einordnung für das Projekt

KeypointDETR ist die modernste Methode in der betrachteten Auswahl. Sie eignet sich sehr gut zur Darstellung des aktuellen Stands der Technik, insbesondere weil sie End-to-End-Training,

Transformer und bipartites Matching kombiniert. Für die tatsächliche Umsetzung sollte jedoch geprüft werden, ob Datenvorverarbeitung und Training innerhalb des Projektzeitraums realistisch sind. Wenn der Fokus auf einem gut erklärbaren und schnell reproduzierbaren Verfahren liegt, ist Skeleton Merger oder SC3K möglicherweise praktischer. Wenn der Fokus auf einem modernen supervised Ansatz liegt, ist KeypointDETR besonders interessant.

9 FL3K: A Fast and Lightweight 3D Keypoint Detector

9.1 Repository und praktische Nutzung

GitHub-Repository: <https://github.com/zhuanjia113/FL3K>

Das Repository zu FL3K stellt laut README Demo-Code zur Keypoint-Detection bereit. Die sichtbare Struktur ist sehr kompakt und überwiegend MATLAB-basiert:

```
FL3K/  
|-- include/  
|   '-- jsonlab-master/  
|-- GetKeypoint_SalientMap.m  
|-- GetSalientValue_PointCloud.m  
|-- Get_3DKeypoint_FL3K.m  
|-- Get_3DKeypoint_IOU.m  
|-- Get_NonLinear_Suppression.m  
|-- test_keypointnet_demo.m  
'-- README.md
```

Wichtig ist: FL3K ist kein klassisches tiefes neuronales Modell mit umfangreichem Nachtraining. Das Repository enthält Demo-Code für die Detektionsfunktion. Für die Projektanforderung „Modell nachtrainieren“ ist FL3K daher nur eingeschränkt geeignet. Es kann aber als schnelle, leicht erklärbare Baseline oder als aktuelle Methode zur Diskussion von Effizienz dienen.

9.2 Problemstellung

FL3K adressiert die Frage, wie 3D-Keypoints schnell und ohne komplexen Lernprozess aus Punktwolken gefunden werden können. Viele learning-based Methoden benötigen aufwendiges Training, große Modelle und teilweise komplexe Datenvorbereitung. FL3K setzt dagegen auf Saliency-Berechnung in Punktwolken [7].

Die Grundannahme ist, dass gute Keypoints geometrisch und regional auffällig sind. Ein Punkt ist dann relevant, wenn er lokal eine wichtige Struktur markiert und zugleich in einer global oder regional bedeutenden Objektregion liegt.

9.3 Grundidee des Verfahrens

FL3K berechnet zunächst eine geometrische Saliency. Diese beschreibt, ob ein Punkt durch seine räumliche Lage und lokale Struktur auffällig ist. Dazu wird eine relative Distanzrepräsentation zum Schwerpunkt bzw. zu regionalen Zentren verwendet. Anschließend wird eine regionale Saliency berechnet, die größere Objektbereiche berücksichtigt.

Beide Informationen werden über eine nichtlineare Aggregation zusammengeführt. Danach werden stabile Keypoints aus der resultierenden Saliency Map ausgewählt. Der Ansatz ist damit deutlich näher an klassischen saliencybasierten Verfahren als an tiefen Autoencoder- oder Transformer-Methoden.

9.4 Einordnung für das Projekt

FL3K ist interessant, weil es aktuell, leichtgewichtig und gut nachvollziehbar ist. Es kann genutzt werden, um zu zeigen, dass moderne 3D-Keypoint-Detection nicht zwangsläufig immer ein großes neuronales Modell benötigt. Für eine Umsetzung mit Nachtraining ist FL3K aber weniger passend, da kein klassischer Trainingsprozess im Vordergrund steht. Falls im Projekt eine einfache Baseline benötigt wird, könnte FL3K dennoch nützlich sein: Man kann Keypoints auf KeypointNet berechnen, visualisieren und mit annotierten Keypoints über einfache Metriken vergleichen.

10 Methodische Auswahl für die praktische Umsetzung

Die Aufgabe erfordert nicht, alle betrachteten Verfahren umzusetzen. Sinnvoll ist eine fokussierte Entscheidung für ein Verfahren, das wissenschaftlich begründet, praktisch reproduzierbar und im Projektzeitraum verständlich erklärbar ist. Aus der Sicht dieser Dokumentation ergeben sich drei naheliegende Umsetzungsrichtungen.

10.1 Variante A: Skeleton Merger als Hauptmethode

Skeleton Merger ist gut geeignet, wenn eine anschaulich erklärbare unüberwachte Methode gesucht wird. Die Idee des Skeletts ist visuell verständlich, die Repository-Struktur ist kompakt und die Methode ist direkt mit KeypointNet verknüpft. Außerdem ist sie in der Aufgabenstellung genannt. Eine Umsetzung könnte darin bestehen, eine Kategorie aus KeypointNet auszuwählen, das Modell nachzutrainieren, Keypoints zu visualisieren und gegen menschliche Keypoints zu evaluieren.

10.2 Variante B: SC3K als moderne selbstüberwachte Methode

SC3K ist geeignet, wenn Robustheit gegen Rotation, Rauschen und Decimation im Vordergrund stehen soll. Das Repository beschreibt Training, Test und Datenstruktur recht klar. Die Methode ist moderner als UKPGAN und Skeleton Merger und eignet sich gut für Experimente mit Störungen der Punktwolke. Sie ist allerdings konzeptionell etwas abstrakter, weil mehrere gekoppelte Verluste zusammenwirken.

10.3 Variante C: KeypointDETR als moderner supervised Ansatz

KeypointDETR ist geeignet, wenn ein aktueller Transformer-Ansatz mit direkter Nutzung von KeypointNet-Annotationen umgesetzt werden soll. Der Vorteil ist die Nähe zum aktuellen Stand der Technik. Der Nachteil ist die komplexere Architektur und Vorverarbeitung. Für ein begrenztes Projekt sollte diese Methode nur gewählt werden, wenn das Repository schnell lauffähig gemacht werden kann.

10.4 Empfohlene pragmatische Vorgehensweise

Für das Projekt erscheint folgende Strategie realistisch:

1. Zunächst KeypointNet lokal vorbereiten und eine Objektkategorie auswählen.
2. Eine einfache Visualisierung der Punktwolken und Ground-Truth-Keypoints erstellen.
3. Skeleton Merger oder SC3K als Hauptmethode nachtrainieren.
4. Die Ergebnisse qualitativ visualisieren und quantitativ mit KeypointNet-Annotationen bewerten.
5. Optional FL3K als schnelle Baseline ohne Training ausführen.
6. KeypointDETR in der Präsentation als moderne supervised Alternative einordnen, falls keine Zeit für eine vollständige Umsetzung bleibt.

11 Evaluation und Bewertungskriterien

11.1 Qualitative Evaluation

Die qualitative Evaluation ist für 3D-Keypoint-Detection besonders wichtig. Punktwolken und vorhergesagte Keypoints sollten gemeinsam dargestellt werden. Dabei sollte geprüft werden, ob die Punkte auf sinnvollen Objektregionen liegen, ob sie das Objekt gut abdecken und ob sie zwischen ähnlichen Objekten konsistent wirken.

Mögliche Visualisierungen sind:

- Punktwolke mit Ground-Truth-Keypoints,
- Punktwolke mit vorhergesagten Keypoints,
- Überlagerung von Ground Truth und Prediction,
- Vergleich mehrerer Objekte derselben Kategorie,
- Robustheitstest bei Rotation, Rauschen oder Subsampling.

11.2 Quantitative Evaluation

Für die quantitative Bewertung kommen mehrere Metriken infrage. Welche davon sinnvoll sind, hängt vom gewählten Verfahren ab.

Lokalisierungsfehler Mittlerer Abstand zwischen vorhergesagten Keypoints und nächstgelegenen Ground-Truth-Keypoints.

PCK / Correct Keypoints Anteil der Keypoints, die innerhalb eines bestimmten Abstandsschwellwerts zu einem Ground-Truth-Keypoint liegen.

mIoU Besonders bei geodesischen Schwellwerten kann gemessen werden, wie stark vorhergesagte und annotierte Keypoint-Regionen überlappen.

Coverage Bewertung, ob die Keypoints das Objekt räumlich gut abdecken.

Semantic Consistency Bewertung, ob Keypoint-Indizes über Objekte derselben Kategorie hinweg ähnliche Objektregionen markieren.

Robustheit Messung der Veränderung der Vorhersagen bei Rotation, Rauschen oder reduzierter Punktzahl.

Laufzeit Besonders relevant, wenn ein Verfahren später in einer Produktions- oder Robotikumgebung genutzt werden soll.

11.3 Reproduzierbarkeit

Für eine wissenschaftliche Projektabgabe sollten alle Experimente reproduzierbar dokumentiert werden. Dazu gehören:

- Git-Commit oder Stand des verwendeten Repositories,
- verwendete Objektkategorien,
- Datenpfade und Datenstruktur,
- Train-/Val-/Test-Splits,
- Hyperparameter,
- Anzahl der Epochen,
- verwendete Hardware,
- zufällige Seeds,
- Evaluationsergebnisse und Visualisierungen.

12 Fazit

Die betrachteten Verfahren zeigen unterschiedliche Wege zur 3D Keypoint Detection. 3DFeat-Net ist eine wichtige Grundlage für schwach überwachtes gemeinsames Lernen von Detektor und Descriptor, ist aber für KeypointNet nur bedingt direkt passend. UKPGAN formuliert Keypoint Detection als selbstüberwachte Informationskompression und ist durch die Rekonstruktionsidee methodisch interessant. Skeleton Merger erweitert diese Idee um eine Skelettstruktur und ist dadurch besonders gut erklärbar und für KeypointNet relevant. SC3K ist ein moderner selbstüberwachter Ansatz, der Robustheit und semantische Kohärenz unter Störungen betont. KeypointDETR steht für eine aktuelle supervised Transformer-Richtung mit End-to-End-Set-Prediction. FL3K zeigt schließlich, dass schnelle saliencybasierte Detektion ohne komplexes Training ebenfalls eine relevante Richtung ist.

Für die praktische Umsetzung empfiehlt sich, nicht zu viele Verfahren parallel zu verfolgen. Aus aktueller Sicht sind Skeleton Merger und SC3K die stärksten Kandidaten für eine realistische Projektumsetzung. Skeleton Merger ist anschaulich und direkt an der Aufgabenstellung orientiert. SC3K ist moderner und gut für Robustheitsexperimente geeignet. KeypointDETR sollte als moderner Stand der Technik in der Dokumentation bleiben, aber nur dann umgesetzt werden, wenn die Datenvorbereitung und das Training frühzeitig erfolgreich getestet werden können.

Literatur

- [1] Yang You, Yujing Lou, Ruoxi Shi, Qi Liu, Yanchao Tai, Lizhuang Ma, Weiming Wang, Cewu Lu. *KeypointNet: A Large-scale 3D Keypoint Dataset Aggregated from Numerous Human Annotations*. CVPR, 2020. Repository: <https://github.com/qq456cvb/KeypointNet>.
- [2] Zi Jian Yew, Gim Hee Lee. *3DFeat-Net: Weakly Supervised Local 3D Features for Point Cloud Registration*. ECCV, 2018. Repository: <https://github.com/yewzijian/3DFeatNet>.
- [3] Yang You, Wenhai Liu, Yanjie Ze, Yong-Lu Li, Weiming Wang, Cewu Lu. *UKPGAN: A General Self-Supervised Keypoint Detector*. CVPR, 2022. Repository: <https://github.com/qq456cvb/UKPGAN>.
- [4] Ruoxi Shi, Zhengrong Xue, Yang You, Cewu Lu. *Skeleton Merger: An Unsupervised Aligned Keypoint Detector*. arXiv:2103.10814, 2021. Repository: <https://github.com/eliphatfs/SkeletonMerger>.
- [5] Mohammad Zohaib, Alessio Del Bue. *SC3K: Self-supervised and Coherent 3D Keypoints Estimation from Rotated, Noisy, and Decimated Point Cloud Data*. ICCV, 2023. Repository: <https://github.com/IIT-PAVIS/SC3K>.
- [6] Hairong Jin, Yuefan Shen, Jianwen Lou, Kun Zhou, Youyi Zheng. *KeypointDETR: An End-to-End 3D Keypoint Detector*. ECCV, 2024. Repository: <https://github.com/bibi547/KeypointDETR>.
- [7] Chengzhuan Yang, Qian Yu, Hui Wei, Fei Wu, Yunliang Jiang, Zhonglong Zheng, Ming-Hsuan Yang. *A Fast and Lightweight 3D Keypoint Detector*. International Journal of Computer Vision, 2025. Repository: <https://github.com/zhuanjia113/FL3K>.